

CONNECTIONS BETWEEN SCHEDULE-FREE OPTIMIZERS, AdEMAMix, AND ACCELERATED SGD VARIANTS

Anonymous authors
Anonymous submission

ABSTRACT

Recent deep learning optimizers, including Schedule-Free optimizers, AdEMAMix, MARS, and Lion, all revise the role of momentum rather than merely retuning standard momentum. In parallel, theoretical work on stochastic gradient descent (SGD) in the noise-dominated regime has shown that acceleration requires decoupling the momentum coefficient from the weight assigned to the current gradient. This paper establishes an explicit bridge between these two developments. We show that several prominent modern optimizers instantiate, approximate, or augment accelerated SGD structure, and we support the resulting interpretation with experiments on a 150m language modeling task. The empirical results identify AdEMAMix, the method most closely aligned with accelerated SGD variants, as the strongest performer in the small-batch regime. Motivated by this connection, we introduce Simplified-AdEMAMix, a streamlined variant that matches AdEMAMix across both large- and small-batch settings while removing the need for two separate momentum terms. The code for Simplified-AdEMAMix will be released upon acceptance.

1 INTRODUCTION

Recent deep learning optimization has produced a series of influential algorithms, including Lion (Chen et al., 2023), ScheduleFreeSGD/AdamW (Defazio et al., 2024), and AdEMAMix (Pagliardini et al., 2024). Although these methods were introduced from different motivations, their common mechanism is unmistakable: each changes how momentum enters the optimizer update.

A separate theoretical literature explains when gradient methods can be accelerated in noisy environments. Classical momentum mechanisms, including heavy-ball and Nesterov momentum, accelerate deterministic gradient descent in settings such as quadratic objectives, but they do not provide acceleration for SGD (Jain et al., 2018; Liu & Belkin, 2020). This limitation led to alternative stochastic acceleration schemes designed specifically for noisy gradients (Jain et al., 2018; Vaswani et al., 2019; Liu & Belkin, 2020; Gupta et al., 2023). A central structural feature of these methods is that they decouple the momentum coefficient from the weight assigned to the current gradient in the optimizer update.

Our central contribution is to make this connection precise. We prove that Schedule-Free SGD is mathematically equivalent to accelerated SGD followed by weight averaging. We further show that Lion, Schedule-Free AdamW, and AdEMAMix can be understood as accelerated-SGD-style updates coupled with preconditioning or related transformations. Although parts of this perspective appear implicitly in prior work (Defazio, 2021), the relationship between these recent practical optimizers and accelerated SGD has not, to the best of our knowledge, been formally established in this unified form.

We validate the theory on a 150m decoder-only transformer trained on 15b tokens with a small batch size of 32k tokens, a setting designed to expose noise-dominated optimization behavior. The results align with the theoretical prediction: Schedule-Free AdamW tracks accelerated SGD-based AdamW (Algorithm 3) closely, and accelerated methods provide a measurable advantage at small batch sizes. We further show that this advantage disappears at sufficiently large batch sizes, precisely where the accelerated-SGD interpretation predicts a weaker benefit.

The paper makes the following contributions:

1. We establish precise theoretical connections between accelerated SGD and recently proposed optimizers, including Schedule-Free SGD and AdEMAMix.

2. We validate the theory on a 150-million-parameter decoder-only transformer by comparing AdamW, Schedule-Free AdamW, AdEMAMix, and MARS. AdEMAMix, the optimizer with the closest structural match to accelerated SGD variants, delivers the strongest performance among these methods.
3. We show that the advantages of these methods diminish at large batch sizes relative to Adam, as expected from their accelerated-SGD structure. In this regime, Adam with momentum scheduling matches the performance of AdEMAMix.
4. We identify a concrete failure mode of Schedule-Free AdamW at high batch sizes: it performs substantially worse than AdamW with cosine decay because Schedule-Free optimizers intrinsically couple the momentum and weight averaging coefficients.
5. We introduce Simplified-AdEMAMix, a modification that preserves AdEMAMix performance across both large and small batch size regimes while eliminating the need for two distinct momentum terms.

2 RELATED WORK

We focus on the strands of prior work that directly motivate our unifying view: accelerated SGD variants and modern optimizers that alter momentum structure.

Jain et al. (2018) introduced an accelerated SGD variant with improved convergence rates for the least-squares problem. Kidambi et al. (2018) sharpened this line of analysis by simplifying the update rule and formally showing that ordinary momentum does not provide acceleration in this setting. Subsequent work (Liu & Belkin, 2020; Vaswani et al., 2019; Gupta et al., 2023) extended stochastic acceleration guarantees to general convex and strongly convex functions under different theoretical assumptions.

Several practical optimizers already contain the same structural ingredients, even when they were introduced from different design principles. Lucas et al. (2019) proposed updates formed from weighted sums of multiple momentum terms with distinct coefficients, and Ma & Yarats (2019) developed an optimizer explicitly inspired by the theoretical framework of Jain et al. (2018). More recently, Chen et al. (2023) introduced a genetic-search-discovered optimizer that, like accelerated SGD variants, assigns different roles to the gradient and the momentum coefficient in the update. Pagliardini et al. (2024) further strengthened this pattern by combining two distinct momentum scales in the final update.

3 BACKGROUND

3.1 MOMENTUM

Momentum is the standard mechanism for accelerating gradient descent in deterministic settings. For weights w_t and momentum coefficient β , the classical update is:

$$m_t = \beta m_{t-1} + \nabla f(w_t); \quad w_t = w_{t-1} - \eta m_t$$

3.2 WEIGHT AVERAGING

Weight averaging is a core tool in stochastic optimization because it reduces iterate noise without changing the underlying gradient estimator. Rather than returning the final iterate w_T , the method returns a weighted average $\bar{w}_T$, with averaging weights denoted by γ_t :

$$\bar{w}_T = (1 - \gamma_T)\bar{w}_{T-1} + \gamma_T w_T$$

All weight-averaging mechanisms analyzed in this paper use coefficients γ_t of the form:

$$\gamma_t \approx 1 - \frac{1}{\delta t}$$

for a constant $0 \leq \delta \leq 1$.

3.3 ACCELERATED SGD

We use the following generalized form as the common template for many accelerated SGD methods:

$$m_t = \beta_{a,t}m_{t-1} + g_t, \quad w_{t+1} = w_t - \eta_{a,t}m_t - \alpha_{a,t}g_t \quad (1)$$

where $\beta_{a,t}, \alpha_{a,t}, \eta_{a,t}$ are possibly time-dependent scalar coefficients, and g_t is the stochastic gradient evaluated at w_t . The subscript ‘a’ marks coefficients belonging to this accelerated SGD parameterization.

Setting $\alpha_{a,t} = 0$ immediately recovers standard SGD with momentum, so the formulation isolates exactly what accelerated SGD adds: a separate current-gradient term. The accelerated SGD algorithms of Jain et al. (2018); Vaswani et al. (2019); Liu & Belkin (2020); Gupta et al. (2023) fall directly within this template; Appendix B gives the explicit reductions.

4 CONNECTIONS BETWEEN EXISTING OPTIMIZERS AND ACCELERATED SGD

This section gives the main theoretical connection: several recent optimizers can be written as accelerated SGD mechanisms, sometimes combined with preconditioning or averaging. The same derivation also motivates Simplified-AdEMAMix, which uses one momentum term yet retains AdEMAMix-level performance across small and large batch size regimes.

4.1 SCHEDULE-FREE SGD

Schedule-Free SGD (Defazio et al., 2024) is a constant-learning-rate optimizer that removes the need for an explicit schedule. Using the notation of Defazio et al. (2024), its update is:

$$\begin{aligned} y_t &= (1 - \beta)z_t + \beta x_t \\ z_{t+1} &= z_t - \gamma g(y_t) \\ x_{t+1} &= (1 - c_{t+1})x_t + c_{t+1}z_{t+1} \end{aligned}$$

Here, y_t is the current model state at which the gradient is evaluated, while x_t is the evaluation iterate.

We expose the accelerated structure by rewriting the dynamics in terms of y_t and a momentum variable m_t , defined as:

$$m_{t+1} = \frac{x_t - z_{t+1}}{\gamma}. \quad (2)$$

This definition simplifies directly to the following recursion:

$$m_t = \frac{x_t - z_{t+1}}{\gamma} \quad (3)$$

$$= \frac{x_t + \gamma g_t - z_t}{\gamma} \quad (4)$$

$$= \frac{x_t - z_t + \gamma g_t}{\gamma} \quad (5)$$

$$= \frac{(1 - c_t)(x_{t-1} - z_t) + \gamma g_t}{\gamma} \quad (6)$$

$$= (1 - c_t)m_{t-1} + g_t. \quad (7)$$

Thus, m_t exactly follows the momentum update in Equation (1) with $\beta_{a,t} = 1 - c_t$. The corresponding update for y_t is:

$$y_{t+1} = (1 - \beta)z_{t+1} + \beta x_{t+1} \quad (8)$$

$$= (1 - \beta)(z_t - \gamma g_t) + \beta((1 - c_{t+1})x_t + c_{t+1}z_{t+1}) \quad (9)$$

$$= (1 - \beta)z_t + \beta x_t - (1 - \beta)\gamma g_t + \beta c_{t+1}(z_{t+1} - x_t) \quad (10)$$

$$= y_t - \gamma[\beta c_{t+1}m_t + (1 - \beta)g_t]. \quad (11)$$

Therefore y_t obeys the weight update in Equation (1) with $\eta_{a,t} = \gamma\beta c_{t+1}$ and $\alpha_{a,t} = \gamma(1 - \beta)$, where $w_t = y_t$. Schedule-Free SGD is consequently an accelerated SGD update in the training iterate. Since the method evaluates using x_t , we next characterize the evaluation dynamics:

$$\begin{aligned} x_{t+1} &= (1 - c_{t+1})x_t + c_{t+1}z_{t+1} \\ x_{t+1} &= (1 - c_{t+1})x_t + c_2 \left(\frac{y_{t+1} - \beta x_{t+1}}{1 - \beta} \right) \\ x_{t+1}(1 - \beta + c_{t+1}\beta) &= (1 - c_{t+1})(1 - \beta)x_t + c_{t+1}y_{t+1} \\ x_{t+1} &= \frac{(1 - c_{t+1})(1 - \beta)x_t + c_{t+1}y_{t+1}}{(1 - c_{t+1})(1 - \beta) + c_{t+1}}. \end{aligned}$$

Thus, x_t is a weighted average of y_t . Recursively expanding x_t shows that it becomes an exponential average of y_t when c_t is constant. This establishes the exact interpretation: Schedule-Free SGD is accelerated SGD followed by weight averaging.

This decomposition attributes the gains of Schedule-Free SGD to two concrete mechanisms:

1. Improved performance relative to standard SGD with momentum, explained by its equivalence to accelerated SGD.
2. Constant-learning-rate training without an explicit schedule, enabled by weight averaging, and in particular by tailed weight averaging; see Section 4.1.3.

Schedule-Free SGD/Adam also delivers two implementation advantages:

- It avoids additional memory for weight averaging.
- It removes the explicit weight averaging coefficient as a hyperparameter.

The same coupling is also a limitation: Section 5.1 shows that tying momentum and weight averaging coefficients together does not scale well to large batch sizes.

4.1.1 CASE: $\beta = 0.0$

When $\beta = 0$, Schedule-Free SGD reduces to standard SGD with weight averaging, as noted in (Defazio et al., 2024). Because $c_t = 1/t$, the method averages from the start of training.

4.1.2 CASE: $\beta = 1.0$

When $\beta = 1$, Schedule-Free SGD becomes standard momentum SGD, with momentum coefficient $\beta_{a,t}$ scaling as $1 - 1/t$, matching the observation in (Defazio et al., 2024).

4.1.3 CASE: $\beta = 0.9$

For the default Schedule-Free SGD setting $\beta = 0.9$, the coefficients have the following direct interpretation:

- Since c_t scales as $1/t$, momentum grows as $1 - 1/t$.
- The ratio between the current-gradient weight and the momentum weight is fixed at $(1 - \beta)/(\beta c_{t+1}) \approx 0.11$.
- Weight averaging effectively covers the most recent 10% of the iterates.

4.2 LION

Lion (Chen et al., 2023) uses the following update rule:

$$m'_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$\theta_t = \theta_{t-1} - \eta \text{sign}(m'_t)$$

$$m_t = \beta_2 m_{t-1} + (1 - \beta_2) g_t.$$

This form gives a direct interpretation of Lion (Chen et al., 2023): it is an accelerated SGD-style update followed by a coordinate-wise sign operation.

4.3 MARS

The practical version of MARS (Yuan et al., 2024), called MARS-Approx, also fits the accelerated SGD framework once its preconditioning step is separated. Ignoring bias correction and clipping, its update is:

$$\begin{aligned} c_t &= g_t + \gamma \frac{\beta_1}{1 - \beta_1} [g_t - g_{t-1}] \\ m_t &= \beta_1 m_{t-1} + (1 - \beta_1) c_t \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) c_t^2 \\ x_{t+1} &= x_t - \eta \frac{m_t}{\sqrt{v_t} + \epsilon} \end{aligned}$$

where m_t and v_t are the first- and second-order momentum terms, respectively, and x_t denotes the model parameters. Rewriting the update with $\hat{m}_t = m_t - \gamma g_t$ yields:

$$\begin{aligned} c_t &= g_t + \gamma \frac{\beta_1}{1 - \beta_1} [g_t - g_{t-1}] \\ \hat{m}_t &= \beta_1 \hat{m}_{t-1} + (1 - \beta_1)(1 - \gamma) g_t \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) c_t^2 \\ x_{t+1} &= x_t - \eta \frac{\hat{m}_t + \gamma g_t}{\sqrt{v_t} + \epsilon} \end{aligned}$$

This formulation makes the accelerated SGD structure explicit in the momentum update. MARS differs from AdamW primarily through its preconditioning choice, and Section 5 evaluates how that distinction affects empirical performance.

4.4 ADEMAMIX

AdEMAMix (Pagliardini et al., 2024) is the closest practical optimizer in our comparison to accelerated SGD-based AdamW. Rather than forming the accelerated SGD update from the current gradient and a single momentum term, AdEMAMix maintains two momentum terms with different coefficients and combines them linearly. Algorithm 1 states the algorithm.

To isolate this structure, we analyze the variant with $\beta_1 = 0$. Pagliardini et al. (2024) showed that this variant is nearly equivalent to the full method at small batch sizes, and our experiments in Section 5 confirm the same behavior. With $\beta_1 = 0$, AdEMAMix aligns with the general accelerated SGD framework in Equation (1). Moreover, the prescribed

schedules for β_3 , the momentum coefficient, and α , which controls the relative weight assigned to the current gradient, closely match theoretical schedules for accelerated SGD (Gupta et al., 2023).

In smooth convex optimization, stochastic acceleration requires a momentum schedule of the form:

$$\beta_{a,t} = 1 - \frac{k}{t}$$

for some constant $k > 0$, as established by Gupta et al. (2023). AdEMAMix approximately implements this requirement by scaling up β_3 over training.

The correspondence is also visible in the momentum normalization. Accelerated SGD maintains momentum in the standard form:

$$m_t = \beta_{a,t} m_{t-1} + g_t$$

whereas Algorithm 1 uses the EMA-style update:

$$m_2^{(t)} \leftarrow \beta_3^{(t)} m_2^{(t-1)} + (1 - \beta_3^{(t)}) g^{(t)}.$$

When $\beta_{a,t}$ scales as $1 - 1/t$, the accumulated contribution of past gradients in accelerated SGD grows proportionally to t . AdEMAMix mirrors this effect because its coefficient α also scales proportionally to t . This close structural match explains why AdEMAMix is the strongest optimizer in the small-batch experiments in Figure 1.

At large batch sizes, the same analysis identifies a limitation. AdEMAMix loses performance when $\beta_1 = 0.0$, consistent with Pagliardini et al. (2024). The theory of Gupta et al. (2023) indicates that the current-gradient weight should decrease at large batch sizes, whereas AdEMAMix keeps a fixed weight of 1 on the current gradient. This mismatch accounts for the observed degradation in that regime.

This analysis motivates the simplified variant introduced next: it keeps an explicit current-gradient weight, removes the need to schedule α , and maintains only a single momentum term. The experiments show that this simpler optimizer matches AdEMAMix in both small- and large-batch setups.

Algorithm 1 Single step of AdEMAMix optimizer.

- 1: **Input:** Data distribution $\mathcal{D}$. Initial model parameters $\theta^{(0)}$. Number of iterations T . Learning rate η . ϵ a small constant. AdamW parameters: β_1, β_2 . AdEMAMix parameters β_3, α . Warmup parameter T_{α, β_3} , note that we usually set it to T . β_{start} is usually set to β_1 .
 - 2: Optional: use schedulers $\eta^{(t)}, \beta_3^{(t)} \leftarrow f_{\beta_3}(t, \beta_3, \beta_{\text{start}}, T_{\alpha, \beta_3})$ and $\alpha^{(t)} \leftarrow f_{\alpha}(t, \alpha, T_{\alpha, \beta_3})$
 - 3: Sample batch: $x \sim \mathcal{D}$
 - 4: Compute gradient: $g^{(t)} \leftarrow \nabla_{\theta} \mathcal{L}_{\theta^{(t-1)}}(x)$
 - 5: Update the fast EMA m_1 : $m_1^{(t)} \leftarrow \beta_1 m_1^{(t-1)} + (1 - \beta_1) g^{(t)}$
 - 6: Update the slow EMA m_2 : $m_2^{(t)} \leftarrow \beta_3^{(t)} m_2^{(t-1)} + (1 - \beta_3^{(t)}) g^{(t)}$
 - 7: Update the second moment estimate: $\nu^{(t)} \leftarrow \beta_2 \nu^{(t-1)} + (1 - \beta_2) (g^{(t)})^2$
 - 8: Update parameters: $\theta^{(t)} \leftarrow \theta^{(t-1)} - \eta^{(t)} \left(\frac{\hat{m}_1^{(t)} + \alpha^{(t)} m_2^{(t)}}{\sqrt{\hat{\nu}^{(t)}} + \epsilon} \right)$
-

4.5 SIMPLIFIED ADEMAMIX

Building on the preceding equivalence, we propose Simplified-AdEMAMix, which removes the second momentum term and eliminates the need to schedule α . Algorithm 2 gives the optimizer, using theory-style momentum rather than EMA-style momentum. The final update assigns a fixed weight α to the gradient; setting $\alpha = 0$ recovers standard Adam after the corresponding transformations of η and β_1 . Section 5 demonstrates that this simpler variant matches AdEMAMix across both small and large batch sizes.

Algorithm 2 Single step of Simplified AdEMAMix optimizer.

- 1: **Input:** Data distribution $\mathcal{D}$. Initial model parameters $\theta^{(0)}$. Number of iterations T . Learning rate η . ϵ a small constant. AdamW parameters: β_1, β_2 . AdEMAMix parameters $\alpha, \beta_{\text{start}}$. Warmup parameter T_{β_1} , note that we usually set it to T .
 - 2: Optional: use schedulers $\eta^{(t)}, \beta_1^{(t)} \leftarrow f_{\beta_1}(t, \beta_1, \beta_{\text{start}}, T_{\beta_1})$
 - 3: Sample batch: $x \sim \mathcal{D}$
 - 4: Compute gradient: $g^{(t)} \leftarrow \nabla_{\theta} \mathcal{L}_{\theta^{(t-1)}}(x)$
 - 5: Update the EMA m_1 : $m_1^{(t)} \leftarrow \beta_1 m_1^{(t-1)} + g^{(t)}$
 - 6: Update the second moment estimate: $\nu^{(t)} \leftarrow \beta_2 \nu^{(t-1)} + (1 - \beta_2) (g^{(t)})^2$
 - 7: Update parameters: $\theta^{(t)} \leftarrow \theta^{(t-1)} - \eta^{(t)} \left(\frac{m_1^{(t)} + \alpha g^{(t)}}{\sqrt{\hat{\nu}^{(t)}} + \epsilon} \right)$
-

5 EXPERIMENTS

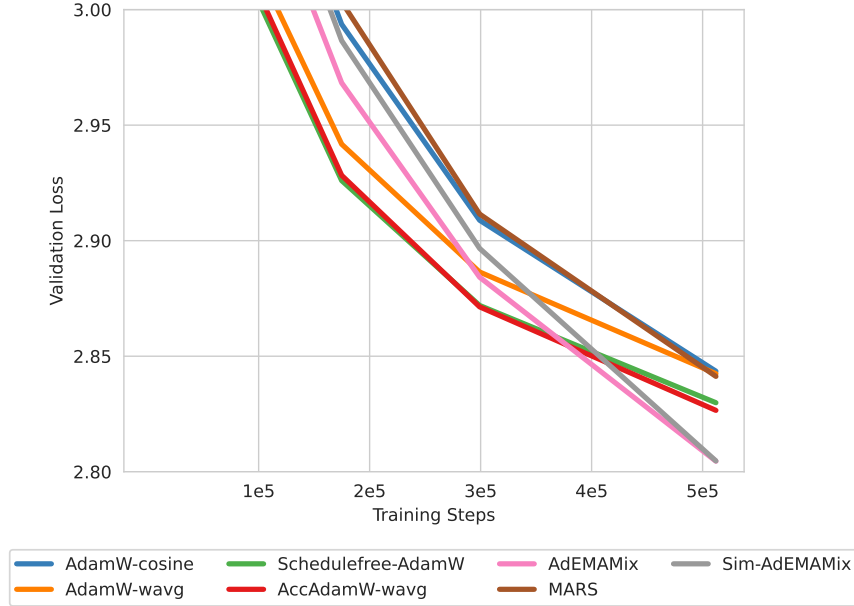


Figure 1: Best-run comparison for the optimizers evaluated in Section 5 on a language modeling task with a decoder-only 150m transformer. AdEMAMix and simplified-AdEMAMix achieve the strongest performance, consistent with their close structural alignment with accelerated SGD variants.

We evaluate the theoretical predictions on a 150-million-parameter decoder-only transformer trained for language modeling on C4. The model uses sequence length 1024 and batch size 32, and is trained for 15 billion tokens ($\approx 5 \times$ Chinchilla), placing the experiment in a noise-dominated optimization regime.

The comparison covers the following optimizers:

1. Standard AdamW with cosine decay
2. Standard AdamW with weight averaging
3. Schedule-Free AdamW
4. Accelerated AdamW with weight averaging (Algorithm 3)
5. MARS

Algorithm 3 Single step of accelerated SGD based Adam with weight averaging. For simplicity we ignore the initialization, other boundary effects such as bias correction, and weight decay. Hyperparameters: Learning rate η , betas = $(\beta_1, \beta_2, \beta_3)$, weight averaging coefficient δ , and epsilon ϵ .

-
- 1: Sample batch B_t .
 - 2: $g \leftarrow -\nabla_w \phi_{B_t}(w_t)$
 - 3: $v \leftarrow \beta_2 v + (1 - \beta_2)(g \odot g)$
 - 4: $N \leftarrow \frac{\beta_3 m + (1 - \beta_3)g}{\sqrt{\hat{v}} + \epsilon}$
 - 5: $w \leftarrow w - \eta N$
 - 6: $m \leftarrow \beta_1 m + (1 - \beta_1)g$
 - 7: $c = \max(1 - 1/t, 1 - 1/(\delta t))$
 - 8: $w_{\text{avg}} \leftarrow c w_{\text{avg}} + (1 - c)w$
-

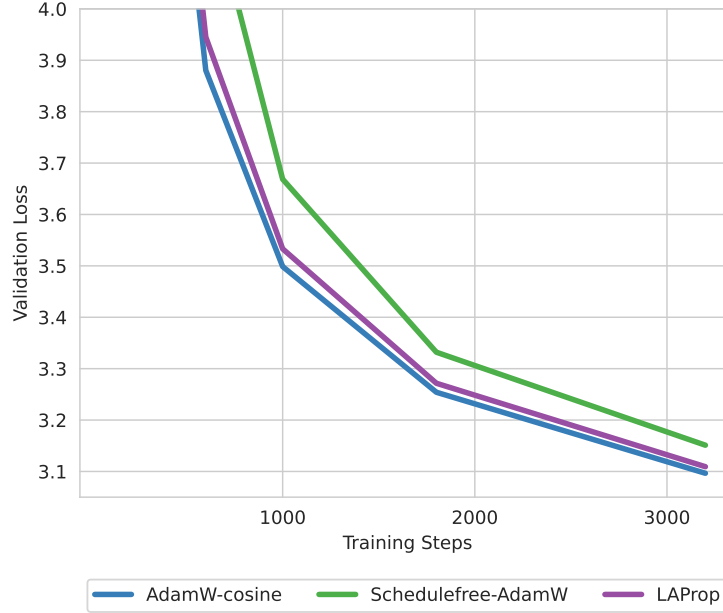


Figure 2: Best-run comparison of AdamW with cosine decay, schedule free AdamW, and LAMProp at higher batch size. The comparison isolates the high-batch-size degradation of Schedule-Free AdamW; Section 5.1 provides the experimental details.

6. AdEMAMix

7. Simplified-AdEMAMix

Section A reports the hyperparameter sweeps used for each optimizer.

Figure 1 confirms the predicted relationship: Schedule-Free AdamW and Accelerated AdamW with tailed weight averaging perform comparably, supporting the theoretical equivalence. Both outperform AdamW with cosine decay and AdamW with tailed weight averaging. AdEMAMix and Simplified-AdEMAMix are the best-performing methods overall, consistent with their closest alignment to accelerated SGD variants.

5.1 LARGE BATCH SIZE EXPERIMENTS

The preceding experiments target the small-batch regime, where gradient noise is central and accelerated-SGD structure is expected to matter. We next move to the large-batch regime to determine whether those advantages persist once

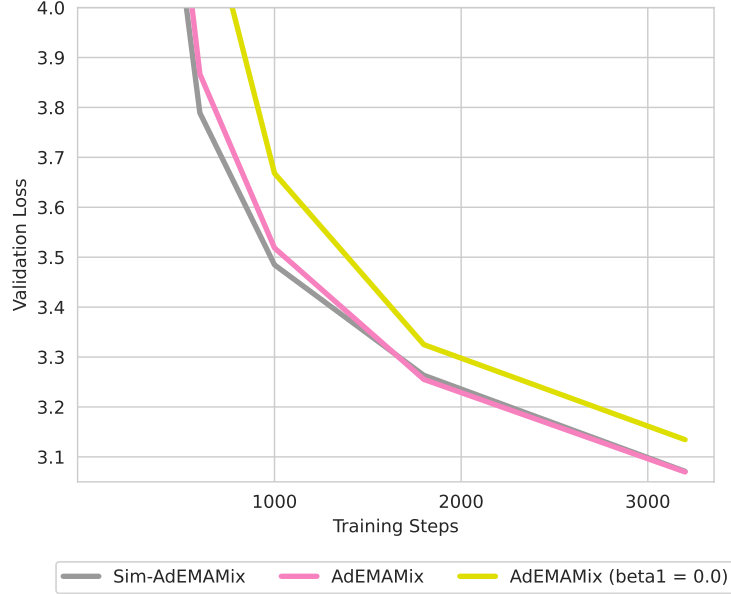


Figure 3: Best-run comparison of AdEMAMix, AdEMAMix with $\beta_1 = 0.0$, and simplified AdEMAMix in the higher-batch-size experiments. The results show that simplified AdEMAMix preserves AdEMAMix performance while the $\beta_1 = 0.0$ variant drops; Section 5.1 provides the experimental details.

gradient noise is reduced. In this setup, models are trained with a batch size of 1 million tokens for 3 billion tokens ($\approx$ Chinchilla scale).

Schedule-Free AdamW: Figure 2 shows a clear degradation: Schedule-Free AdamW performs significantly worse than AdamW. The evidence points to the coupling between weight averaging and momentum coefficients. At higher batch sizes, the optimal momentum value is substantially lower than $1 - 1/t$. A scaling factor $\approx 1 - r/t$ for some $r \geq 1$ can reduce the momentum, but increasing r also shortens the effective weight averaging window.

The order of momentum and preconditioning is another possible explanation, but the experiments rule it out as the main driver. AdamW applies momentum before preconditioning, while Schedule-Free AdamW applies preconditioning before momentum, making it algorithmically similar to LARProp (Ziyin et al., 2021). Yet Figure 2 shows that AdamW and LARProp perform comparably, so this ordering difference does not explain the Schedule-Free AdamW gap.

AdEMAMix: At large batch sizes, Figure 3 confirms the pattern reported by Pagliardini et al. (2024): setting $\beta_1 = 0.0$ causes a substantial performance drop relative to using two momentum terms. The accelerated-SGD perspective explains this behavior. AdEMAMix assigns a fixed weight of 1 to the current gradient, whereas theoretical accelerated SGD variants (Gupta et al., 2023) require the current-gradient weight to diminish as batch size increases.

Figure 3 also shows that Simplified-AdEMAMix matches AdEMAMix while eliminating the second momentum term. In this large-batch setting, the matched performance occurs at $\alpha = 0.0$, so Simplified-AdEMAMix reduces to standard Adam with momentum scheduling.

6 CONCLUSION

This work establishes a unified accelerated-SGD interpretation of several recent optimizers, including Schedule-Free optimizers, AdEMAMix, MARS, and Lion. The theory explains both where these methods should help and where their benefits should weaken. Empirically, AdEMAMix, the method with the closest match to theoretical accelerated SGD variants, achieves the best performance in small-batch training.

The same connection leads directly to Simplified-AdEMAMix, which removes the need for two momentum buffers. Across both small and large batch sizes, Simplified-AdEMAMix matches AdEMAMix while eliminating its additional momentum memory overhead, showing that the accelerated-SGD structure can be retained in a simpler optimizer.

REFERENCES

- Chen, X., Liang, C., Huang, D., Real, E., Wang, K., Pham, H., Dong, X., Luong, T., Hsieh, C., Lu, Y., and Le, Q. V. Symbolic discovery of optimization algorithms. In Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., and Levine, S. (eds.), *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*. URL http://papers.nips.cc/paper_files/paper/2023/hash/9a39b4925e35cf447ccba8757137d84f-Abstract-Conference.html.
- Defazio, A. Momentum via primal averaging: Theoretical insights and learning rate schedules for non-convex optimization, 2021. URL <https://arxiv.org/abs/2010.00406>.
- Defazio, A., Yang, X., Mehta, H., Mishchenko, K., Khaled, A., and Cutkosky, A. The road less scheduled. *CoRR*, abs/2405.15682, 2024. doi: 10.48550/ARXIV.2405.15682. URL <https://doi.org/10.48550/arXiv.2405.15682>.
- Gupta, K., Siegel, J., and Wojtowysch, S. Achieving acceleration despite very noisy gradients, 2023. URL <https://arxiv.org/abs/2302.05515>.
- Jain, P., Kakade, S. M., Kidambi, R., Netrapalli, P., and Sidford, A. Accelerating stochastic gradient descent for least squares regression. In Bubeck, S., Perchet, V., and Rigollet, P. (eds.), *Proceedings of the 31st Conference On Learning Theory*, volume 75 of *Proceedings of Machine Learning Research*, pp. 545–604. PMLR, 06–09 Jul 2018. URL <https://proceedings.mlr.press/v75/jain18a.html>.
- Kidambi, R., Netrapalli, P., Jain, P., and Kakade, S. M. On the insufficiency of existing momentum schemes for stochastic optimization. In *International Conference on Learning Representations*, 2018. URL <https://openreview.net/forum?id=rJTutzbA->.
- Liu, C. and Belkin, M. Accelerating sgd with momentum for over-parameterized learning. In *International Conference on Learning Representations*, 2020. URL <https://openreview.net/forum?id=rlgixp4FPH>.
- Lucas, J., Sun, S., Zemel, R., and Grosse, R. Aggregated momentum: Stability through passive damping. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=Syxt5oC5YQ>.
- Ma, J. and Yarats, D. Quasi-hyperbolic momentum and adam for deep learning. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=SlfUpoR5FQ>.
- Pagliardini, M., Ablin, P., and Grangier, D. The ademamix optimizer: Better, faster, older. 2024. URL <https://arxiv.org/abs/2409.03137>.
- Porian, T., Wortsman, M., Jitsev, J., Schmidt, L., and Carmon, Y. Resolving discrepancies in compute-optimal scaling of language models. *CoRR*, abs/2406.19146, 2024. doi: 10.48550/ARXIV.2406.19146. URL <https://doi.org/10.48550/arXiv.2406.19146>.
- Vaswani, S., Bach, F., and Schmidt, M. Fast and faster convergence of sgd for over-parameterized models and an accelerated perceptron. In Chaudhuri, K. and Sugiyama, M. (eds.), *Proceedings of the Twenty-Second International Conference on Artificial Intelligence and Statistics*, volume 89 of *Proceedings of Machine Learning Research*, pp. 1195–1204. PMLR, 16–18 Apr 2019. URL <https://proceedings.mlr.press/v89/vaswani19a.html>.
- Yuan, H., Liu, Y., Wu, S., Zhou, X., and Gu, Q. Mars: Unleashing the power of variance reduction for training large models, 2024. URL <https://arxiv.org/abs/2411.10438>.
- Ziyin, L., Wang, Z. T., and Ueda, M. Laprop: Separating momentum and adaptivity in adam, 2021. URL <https://arxiv.org/abs/2002.04839>.

A HYPERPARAMETERS

The small-batch experiments use the following hyperparameter sweeps.

1. AdamW with cosine decay and 51.2k warmup: learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.9, 0.95]$, and β_2 in $[0.99, 0.999, 0.99968, 0.9999]$. The best values were $\beta_1 = .9$ and $\beta_2 = .999$, matching the defaults. Although larger batch sizes commonly use $\beta_2 = .95$, the benefit of higher β_2 at smaller batch sizes has also been observed by Porian et al. (2024).
2. AdamW with cosine decay and 10k warmup: learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, with $\beta_1 = 0.9$ and $\beta_2 = 0.999$ fixed to the best values from the previous sweep. This shorter warmup underperformed the 51.2k-step warmup.
3. AdamW with constant-fraction weight averaging: learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, $\beta_1 = 0.9$, β_2 in $[0.99, 0.997, 0.999, 0.9997]$, δ in $[0.05, 0.1, 0.2]$.
4. AdamW with cosine decay and weight averaging: learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, and δ in $[0.025, 0.05, 0.1]$.
5. Accelerated SGD based AdamW with cosine decay: learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.999, 0.99968, 0.9999]$, β_2 in $[0.99, 0.9968, 0.999]$, and $\beta_3 = 0.9$.
6. Accelerated SGD based AdamW with constant learning rate and weight averaging: learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.99684, 0.999]$, β_2 in $[0.999]$, $\beta_3 = 0.9$, and δ in $[0.05, 0.1]$.
7. Accelerated SGD based AdamW with cosine decay and weight averaging: learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.99684, 0.999]$, $\beta_2 = 0.999$, δ in $[0.05, 0.1]$, and $\beta_3 = 0.9$.
8. Schedulefree AdamW with constant learning rate: learning rate in $[3.16e-4, 1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.8, 0.9, 0.95]$, and $\beta_2 = 0.999$.
9. Schedulefree AdamW with cosine decay: learning rate in $[3.16e-4, 1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.8, 0.9, 0.95]$, and $\beta_2 = 0.999$.
10. MARS: learning rate in $[3.16e-4, 1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.9, 0.95, 0.99]$, β_2 in $[0.99, 0.999]$, and γ in $[0.0, 0.01, 0.02, 0.03, 0.04, 0.05]$, with precondition 1d set to True.
11. AdEMAMix: learning rate in $[3.16e-4, 1e-3, 3.16e-3]$, β_1 in $[0.0, 0.9]$, $\beta_2 = 0.999$, β_3 in $[0.99, 0.999, 0.9999]$, and α in $[2, 4, 8, 16]$.
12. Sim-AdEMAMix: learning rate in $[1e-6, 3.16e-6, 1e-5, 3.16e-5]$, β_1 in $[0.99, 0.999, 0.9999]$, $\beta_2 = 0.999$, and α in $[10, 20, 50, 100]$.

The large-batch experiments use the following sweeps:

1. Schedule-Free AdamW: learning rate in $[1e-3, 3.16e-3, 1e-2]$, β in $[0.8, 0.9, 0.95]$, β_2 in $[0.9, 0.95]$, and r in $[0.0, 5.0, 9.0, 50.0]$.
2. AdamW: learning rate in $[1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.9, 0.95]$, and β_2 in $[0.9, 0.95]$.
3. LProp: learning rate in $[1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.9, 0.95]$, and β_2 in $[0.9, 0.95]$.
4. AdEMAMix: learning rate in $[1e-3, 3.16e-3, 1e-2]$, β_1 in $[0.0, 0.9]$, $\beta_2 = 0.95$, β_3 in $[0.9, 0.95, 0.99]$, and α in $[2, 4, 8, 16]$.
5. Sim-AdEMAMix: learning rate in $[1e-4, 3.16e-4, 1e-3]$, β_1 in $[0.9, 0.95, 0.99]$, $\beta_2 = 0.95$, and α in $[0.0, 0.5, 1.0]$.

B EQUIVALENCE OF PREVIOUS ACCELERATION METHODS

Equation (1) gives the accelerated SGD template used throughout the paper. This appendix shows explicitly that the methods of Jain et al. (2018); Vaswani et al. (2019); Liu & Belkin (2020); Gupta et al. (2023) all fall within that form.

B.1 AGNES

The update of Gupta et al. (2023) is:

$$x'_n = x_n + \alpha v_n \quad x_{n+1} = x'_n - \eta g'_n \quad v_{n+1} = \rho_n(v_n - g'_n)$$

where g'_n is the stochastic gradient evaluated at x'_n , while the final function is evaluated at x_n . These equations rewrite as

$$x'_{n+1} = x'_n - \eta g'_n + \alpha v_{n+1} \quad -\frac{v_{n+1}}{\rho_n} = \rho_{n-1} \left(-\frac{v_n}{\rho_{n-1}} \right) + g'_n$$

Thus x'_{n+1} follows an update of the form in Equation (1).

B.2 ASGD

The update of Jain et al. (2018) is:

$$y_{j-1} = \alpha x_{j-1} + (1 - \alpha)v_{j-1} \quad x_j = y_{j-1} - \delta g_{j-1} \quad z_{j-1} = \beta y_{j-1} + (1 - \beta)v_{j-1} \quad v_j = z_{j-1} - \gamma g_{j-1}$$

where g_{j-1} is the stochastic gradient evaluated at y_{j-1} and the function is evaluated on the tail-averaged x .

These update equations rewrite as:

$$y_j = y_{j-1} - \alpha \delta g_{j-1} - (1 - \alpha)[y_{j-1} - v_j] \quad \frac{y_{j-1} - v_j}{\gamma - (1 - \beta)\alpha\delta} = (1 - \beta)\alpha \frac{y_{j-2} - v_{j-1}}{\gamma - (1 - \beta)\alpha\delta} + g_{j-1}$$

The resulting dynamics follow the form of Equation (1).

B.3 MASS

The update of Liu & Belkin (2020) is:

$$w_{t+1} = u_t - \eta_1 g_t \quad u_{t+1} = (1 + \gamma)w_{t+1} - \gamma w_t + \eta_2 g_t$$

where g_t is the stochastic gradient evaluated at u_t and the function is evaluated at w_t . These equations rewrite as:

$$u_{t+1} = u_t - \gamma(w_t - u_t) - [\eta_1(1 + \gamma) - \eta_2]g_t \quad \frac{w_t - u_t}{\eta_1\gamma - \eta_2} = \gamma \frac{w_{t-1} - u_{t-1}}{\eta_1\gamma - \eta_2} + g_t$$

The rewritten dynamics therefore follow Equation (1).

B.4 SGD WITH NESTEROV ACCELERATION

The update of Vaswani et al. (2019) is:

$$w_{k+1} = \zeta_k - \eta g_k \quad \zeta_k = \alpha_k v_k + (1 - \alpha_k)w_k \quad v_{k+1} = \beta_k v_k + (1 - \beta_k)\zeta_k - \gamma_k \eta g_k$$

These equations rewrite as:

$$\zeta_{k+1} = \zeta_k - \eta g_k + \alpha_{k+1}[v_{k+1} - w_{k+1}]; \quad v_{k+1} - w_{k+1} = \beta_k(1 - \alpha_k)[v_k - w_k] - \eta(\gamma_k - 1)g_k$$